

Simulator Enhancements - Krittika - SIMD units and Network on Chip

Manish Manchali Lohith Artham Somesh Nawandar Suryaa Senthilkumar Shanthi Sreemanth Prathipati

Abstract

Deep Neural Networks have been used widely for multiple applications recently. Convolutional Neural Networks are a subclass of a DNNs that have been used for image detection and classification. Systolic array-based DNN accelerators are well-suited for CNNs. State-of-the-art DNN models are too large to fit on a single-core accelerator, hence leading to increased adoption of multi-core systolic array-based accelerators. *Krittika* is a multi-core systolic array simulator, that uses SCALE-SIM as its backend to enable cycle accurate analysis of DNN models. There are multiple considerations when mapping a model onto a multi core accelerator - such as the communication overhead between multiple cores based on their network topology, the impact of network parameters such as latency and bandwidth, and model partitioning schemes across these cores. In order to facilitate and accelerate the design of multi-core systolic array accelerators, we are augmenting *Krittika* to consider these factors while calculating performance metrics. Further, to allow such distributed analysis of DNN models, we are extending *Krittika* to support more SIMD non-linear functions such as GeLU, *tanh* and so on. This work will further increase *Krittika*'s ability to shape novel accelerator design and exploration.

1 Introduction

Deep Neural Networks (DNNs) have become increasingly crucial over the last few years in areas of object detection, chatbots, disease detection and diagnosis, recommendation models for streaming services, and robotics. Out of these, Convolutional Neural Networks are the most common and the introduction of AlexNet[10] started a race of Deep learning across the scientific community. The cessation of performance scaling in CPUs, coupled with the substantial power requirements of GPUs required for DNNs training, has resulted in a flood of specialized deep neural network (DNN) accelerators emerging from both academic and industrial spheres.

A distributed machine learning system simulator plays a crucial role in facilitating a systematic study of challenges inherent in modern deep learning systems. Simulating the behavior of distributed computing environments, allows researchers to explore various aspects such as scalability, fault tolerance, identifying bottlenecks, and other performance optimization in a controlled and reproducible manner. Through experimentation within simulated environments, developers can assess the impact of different configurations, algorithms, and architectures on the overall performance of distributed

machine learning systems. Moreover, these simulators enable the testing of innovative strategies for addressing challenges like communication overhead, resource deadlocks/livelocks, and synchronization bottlenecks, thus providing valuable insights for the design and optimization of real-world distributed deep learning systems.

Among the array architectures proposed for DNN accelerators, 2D array configurations have gained prominence due to their ability to enable operand reuse along two dimensions. Within this category, systolic array architectures are well-suited for CNNs because their local data shifting mirrors the inherent data flow of 2D convolutions. Systolic arrays also excel at efficiently handling matrix-matrix and matrix-vector operations crucial for DNN training and executing LSTMs, respectively. Their efficiency lies in the localized operand movement, ensuring high compute density, low energy consumption, and simplified control. When combined with a carefully crafted memory hierarchy, systolic array architectures effectively exploit the abundant data reuse in DNNs while maintaining high throughput by keeping processing elements active. Therefore understanding the importance of systolic arrays we consider the work Scalesim [3] and *Krittika* and give an overview of them and augment *Krittika* with analytical NoC capabilities and add support for more SIMD non-linear activation functions.

1.1 Key Contribution

This project makes two primary contributions to *Krittika*:

- Adding support for additional SIMD activation functions
- Integration of an analytical NoC model.

These improvements aim to improve the distributed systolic array-based simulator, *Krittika*, by integrating communication overheads. Furthermore, introduce additional SIMD units to facilitate the execution of various models for achieving cycle-accurate performance. This endeavor facilitates a deeper comprehension of distributed deep learning acceleration.

2 Background and Motivation

2.1 DNN Basics

Deep Neural Networks (DNNs) are composed of multiple layers of interconnected nodes, designed to model complex patterns and relationships in data. Each layer typically consists of neurons organized into layers, such as convolutional layers for image processing or recurrent layers for sequential data. DNNs are trained using large datasets through a

process called backpropagation, where the network adjusts its parameters to minimize the difference between predicted and actual outputs. With the advent of specialized hardware accelerators, such as GPUs, TPUs, and FPGAs, DNN computations can be performed much faster and more efficiently, enabling real-time inference and training tasks. These accelerators leverage parallel processing and optimized architectures to execute DNN operations with high throughput and low latency, revolutionizing the field of deep learning and enabling the development of sophisticated AI applications.

DNN simulators are essential for evaluating neural network models and hardware accelerators. They enable researchers to assess performance metrics and optimize designs. Cycle-accurate simulations provide precise insights into system behavior and performance.

2.2 SCALE-SIM

SCALE-SIM is a cycle-accurate simulator for DNN accelerators called Systolic Accelerator Simulator, [3] which models compute performance, memory accesses both on-chip and off-chip, also models interface bandwidth information for a given neural network. SCALE-SIM has the following two basic building blocks:

- A systolic array-based compute unit, with input parameters of size and ratio.
- A simplified memory sub-system with double buffered SRAM memories (size is user controlled), one buffer each for two operands and one output of operation.

The inputs to the tool are the layer dimensions of a given neural network workload, and the hardware architecture parameters. SCALE-SIM can model both scaleup i.e. monolithic or one partition and scale-out i.e. with multiple partition instances.

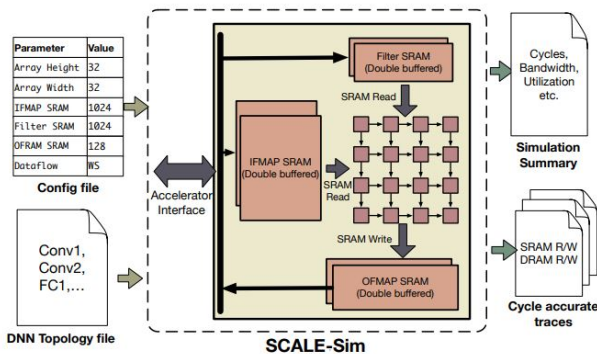


Figure 1. Overview of SCALE-SIM

Currently, SCALE-SIM models a single systolic array which is not sufficient to model complex models as they require

distributed computing to handle the increased size requirements.

2.3 Krittika

The increasing scale and complexity of deep learning models, along with the explosion of big data, underscore the necessity for distributed machine learning accelerator simulators. As models grow larger and datasets become more extensive, distributed ML simulators are essential for efficiently handling the heightened computational workload and memory demands. *Krittika*, a distributed deep-learning accelerator simulator, is constructed atop SCALE-SIM. It instantiates multiple systolic arrays to enable distributed execution and supports nonlinear function latency modeling such as RELU. Its input formats are very similar to SCALE-SIM. While SCALE-SIM handles intra-layer scheduling (e.g., ws, is, os, etc.), inter-layer scheduling is currently not modeled in *Krittika*, defaulting to the layer-sequential pattern.

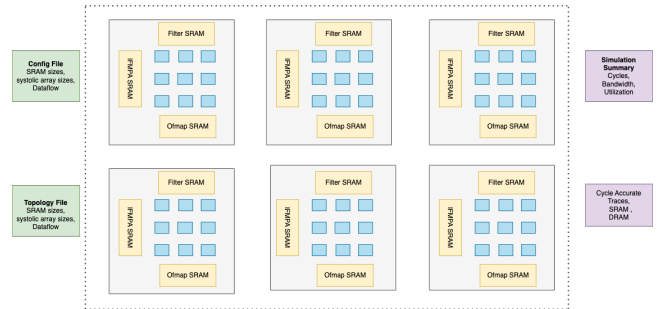


Figure 2. Overview of Krittika

2.4 Inter-layer Scheduling

The expansion of DNN accelerator scale underscores the importance of inter-layer scheduling, which involves allocating computing resources to each layer and determining the computing order within a DNN. As the scale of accelerators increases, the impact of inter-layer scheduling becomes more significant [8].

2.4.1 Layer Sequential (LS) Pattern. In the LS scheme, every node within the accelerator grid sequentially processes its designated layer, ensuring no overlap between layers. This straightforward approach dedicates all computing resources to one layer at a time, simplifying synchronization and resource allocation with its deterministic execution order.

2.4.2 Layer Pipeline (LP) Pattern. LP distributes HW tiles among layers in a pipeline fashion, enabling concurrent processing. This boosts throughput and resource utilization but requires intricate synchronization mechanisms.

In summary, LS processes layers sequentially without overlap, while LP orchestrates layers in a pipeline manner, enabling concurrent processing and higher throughput. For

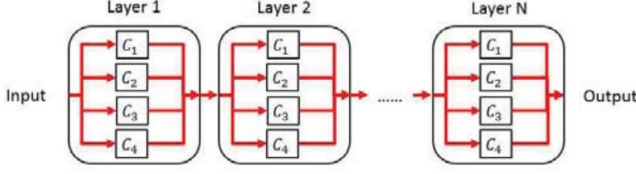


Figure 3. Layer-Sequential Pattern [13]

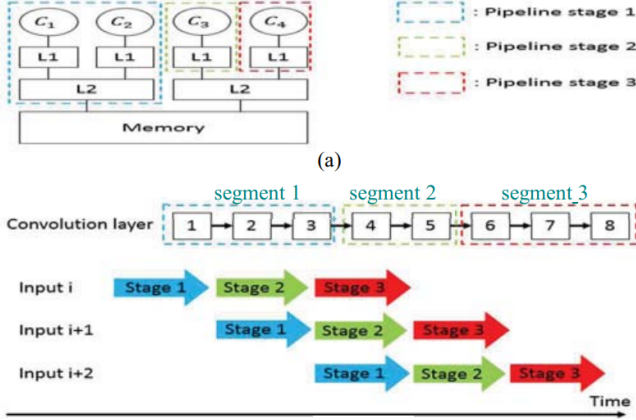


Figure 4. Layer-Pipeline Pattern [13]

example, Fused-layer[2] allocates different computing resources for layers and uses LP, reducing DRAM access by 95% compared to LS. LP is advantageous with memory-bound models, emphasizing the need for a robust NoC to prevent communication bottlenecks, especially with large DNN state of the art models. These schemes will guide our evaluation of NoC integration into Kritika.

Integrating a NoC into *Kritika* is motivated by the crucial role of communication infrastructure in modern distributed computing. NoCs facilitate efficient data exchange among nodes, crucial for parallel processing. *Kritika*'s limited network-level interaction hinders insights into communication overhead.

2.5 Analytical NoC Model

2.5.1 Network-on-Chip (NoC) in Deep Learning. Traditional accelerators like CPUs, GPUs, ASICs, and FPGAs face limitations with large-scale DNN models due to power consumption and computational inflexibility. NoC integration in DNN accelerators separates data computation and communication, offering better flexibility and simplicity. Neuron operations are executed by processing elements (PEs), and results are propagated through packets to the next PE, enabling efficient execution of different DNN models. NoC-based designs allow adjustments in computational power and performance, with proper mapping and routing algorithms optimizing resource utilization. This approach reduces off-chip memory

accesses and facilitates data exchange between PEs through a flexible interconnect.

2.5.2 NoC based DNN Accelerator. NoC simplifies interconnection complexity in DNN accelerators, necessitating high-level NoC-based neural network simulators. Given the scale of state-of-the-art and future systems, which may involve thousands of NPUs, cycle-level simulation with Garnet[1] becomes impractical due to its slow speed. The AstraSim analytical NoC model[12] is an approach that applies mathematical and analytical tools to build and optimize network-on-chip (NoC) topologies.

ANoC estimates communication delay using a simple equation based on link latency, hops, message size, and link bandwidth. It supports a variety of network topologies (namely Ring, Fully Connected and Switch) and congestion aware and unaware routing, and allows configuring the rest of the network parameters like inter-node bandwidth, latency and number of nodes. This approach is suitable for bandwidth-bound communication scenarios, such as those involving large communication sizes. The analytical equation can be adapted to consider other effects to capture complex system and network optimizations.

2.6 SIMD units

SIMD stands for "Single Instruction, Multiple Data." SIMD units are specialized processing units or instruction sets found in modern processors that enable parallel processing of multiple data elements using a single instruction. SIMD architectures exploit data-level parallelism by performing the same operation on multiple data elements simultaneously. SIMD units are commonly utilized to accelerate the computation of activation functions in deep learning models. Activation functions, such as ReLU, tanh, and others, are typically applied element-wise to the outputs of neural network layers.

Activation functions play a crucial role in neural network layers, introducing non-linearities essential for learning complex patterns in data. They are applied during both training and inference stages, impacting model convergence and training time. Accurately modeling activation function latency in deep learning simulators is vital for precise performance predictions and optimizing model training. It also facilitates architectural exploration, providing insights into activation function implementations and hardware accelerators, thus advancing deep-learning hardware design and system efficiency.

Integrating SIMD support for various non-linear operators (such as softmax, maxpool, sigmoid, etc.) in *Kritika* enables thorough analysis of deep learning systems, allowing for cycle-accurate simulations with diverse non-linear functions. Furthermore, this support enables the simulation of a wider range of models, including popular architectures

like AlexNet[10] and BERT[7], enhancing the versatility and applicability of *Krittika* in deep learning research and development.

3 Related Work

Several deep learning accelerator simulators have been developed in academia over the years. Astra-Sim[12] stands out as a distributed machine learning simulator designed to systematically address challenges within modern deep learning systems. It enables the exploration of bottlenecks and the creation of efficient methodologies for handling large DNN models across various future platforms. It has a plug-and-play API model wherein you can use any network, compute, and memory simulator backends. Here, in our work, we utilize Astra-Sim’s NoC Analytical Model to integrate into *Krittika* and provide cycle-accurate analysis for design space exploration which is not the case for Astra-Sim.

4 Evaluation

In this section, we will initially present our planned methodology of how we intend to add an analytical NoC model into *Krittika*, and how we intend to add support for multiple new non-linear operator SIMD units. After this, we present our intended evaluation metrics.

4.1 Methodology

4.1.1 Extending *Krittika* to support inter-core communication. Currently, while *Krittika* supports partitioning the model across the multiple cores, it does not factor the inter-core communication delay into its performance metrics. As part of this project, we would have to model this network latency to enable more accurate performance analysis of distributed DNN models. We intend to extend *Krittika* to take a network topology input alongside a config file to describe the cores and a topology file to describe the layers of the workload topology. Figure 14 pictorially summarizes the inputs to the simulator.

The user will need to pass information such as the topology between cores (like bus, ring, fully connected for instance), the number of cores connected, and the inter-core latency and peak bandwidth. After partitioning the layer across cores using LS and LP partitioning schemes, *Krittika* will then extract the relevant per-core topology information, validate it, and then internally interface with the analytical NoC simulator[12] to get the latency for this inter-core communication and account for this latency in performance numbers.

Layer-Sequential Implementation In this case, we use 4 layers and 4 cores, applying Layered Sequential (LS) as shown in 5. *Krittika* supports input and filter partitioning. Here, the input data is divided into 4 parts, so each core receives the same filters loaded from DRAM and different segments of the input data. This way, each core processes

a portion of the input feature map (ifmap) according to the following code:

```
foreach core i:
    start_index = i * (ifmap_size / total_cores)
    end_index = (i + 1) * (ifmap_size / total_cores)
    core_to_input = ifmap[start_index : end_index]
```

The processing involves three DRAM interactions: reading the input feature map (ifmap), reading the filters (f), and writing the output feature map (ofmap). The cores operate simultaneously, each handling a different segment of the input data.

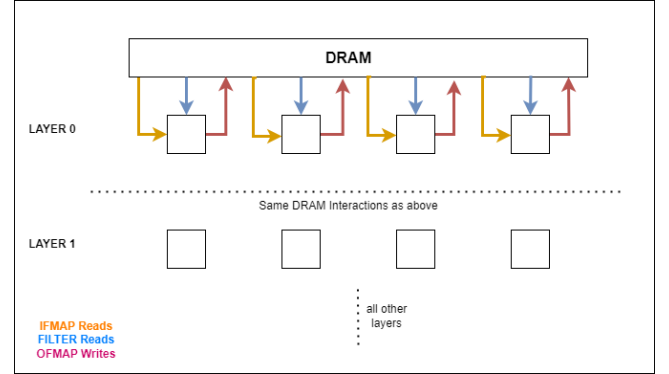


Figure 5. LS with 4-way partitioning across 4 cores

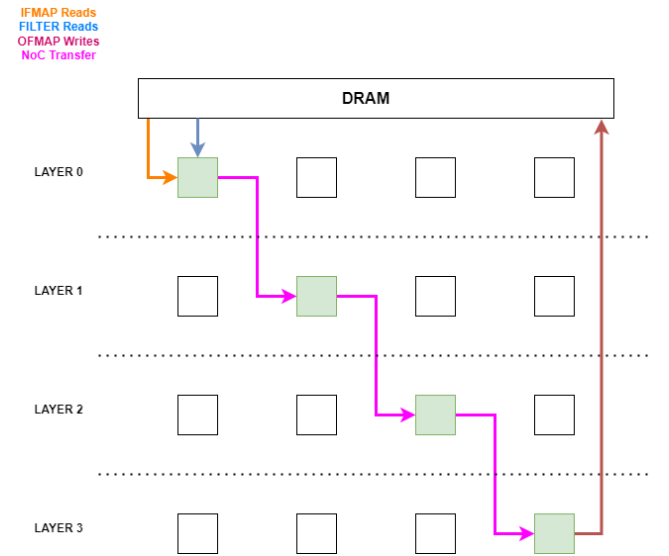


Figure 6. LP with each core working on a unique layer

Layer-Pipeline Implementation In this scenario, we are working with 4 identical layers (for simplicity sake of this explanation but supports different) and 4 cores, where Layered Pipeline (LP) is applied as shown in 6. Each core processes a layer sequentially:


```

foreach core i:
  if core == 0 : core_to_input = ifmap
  else : core_to_input = ofmap of core <i-1>

```

The process involves three DRAM interactions: reading the input feature map (ifmap) for the first core, reading the filters for each layer, and writing the output feature map (ofmap) from the last layer or core. The cores work sequentially, with one core active at a time processing different layers. Compared to LS, LP reduces DRAM interaction but introduces NoC transfer latency that must be considered in cycle computations.

A naive implementation of LP is not ideal as you would have to wait for the whole layer to finish and then transfer over the data (Figure 7). However, a better-suited version to implement LP would be to execute each layer on a more tile-based granularity approach (Figure 8). We execute each tile of each layer and then push the tile over to the next processing layer. This approach has multiple benefits. It lowers the total execution time and increases each instance's effective utilization as we can compute in parallel. However, the important assumption here is you shouldn't have a nonlinear operator following because partial products can't be sent over. To make implementation more feasible, we decided to go with Output Stationary Dataflow which guarantees the generation of the output products. So all of our results section is with the Output Stationary Dataflow model.

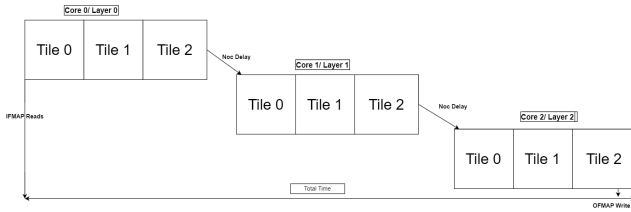


Figure 7. Naive LP Approach

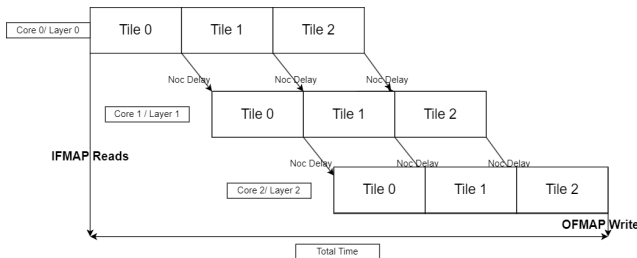


Figure 8. Tiled Based Approach

4.1.2 Analytical Model. We've crafted an analytical model that serves as a lightweight exploration tool. It doesn't delve into memory traces, making it a lightweight tool for exploration. This model serves as a reference in a general context

for making modifications to the Krittika codebase. If computing resources allow simulating 10+ cores, the graphs in this subsection illustrate the trend expected with our Analytical NoC integration to Krittika.

As DNN models become more intricate, providing direct DRAM access to each core isn't feasible. By incorporating Network-on-Chip (NoC) integration, Krittika enables cores to access DRAM indirectly through interconnections. This approach fosters data reuse and minimizes the need for accessing DRAM directly, a key factor in optimizing resource usage and achieving peak performance in large-scale DNN computations.

To illustrate the importance of NoC support, we've used MobileNet as a case study in our analytical model. We've simulated a configuration with 27 cores, where the NoC is 10 times faster than DRAM latencies. By assuming that SRAMs are sufficiently large to hold filter maps without fetching from DRAM, we underscore the significance of NoC. We've explored Layer-Sequential (LS) with weight stationary dataflow in two setups: one with each core having exclusive DRAM access, and the other with only one DRAM port shared among all cores. This comparison highlights the essential role of NoC in multi-core systems. These configurations are chosen because large DNN models demand multi-core systems, and allocating a dedicated DRAM port for each core is impractical and lacks scalability. NoC enables communication between cores connected to DRAM, facilitating data retrieval from DRAM through intermediary cores.

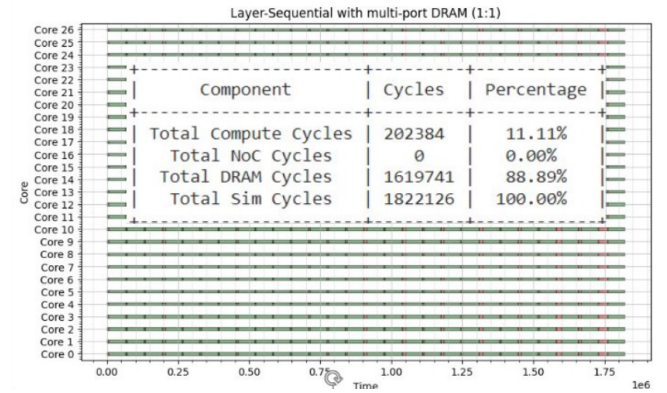


Figure 9. MobileNet on 27 cores with LS and 'ws' dataflow. Compute(Red), DRAM(Green)

Our analysis extends to Layer-Pipeline (LP) with output-stationary dataflow on 27 cores. Here, each core handles one layer, and data is pipelined across cores. This strategy enhances data reuse, reduces DRAM accesses resulting in improved performance and resource utilization. The graphs demonstrate the distribution of cycles across DRAM, NoC, and compute tasks, showcasing the efficiency of LP compared to different the LS configurations discussed previously. The

table 1 demonstrates that LP is twice as slow compared to the multi-port scenario, but it is 12 times faster than the single-port case. This underscores the necessity of a NoC in multi-core systems to enhance data reuse.

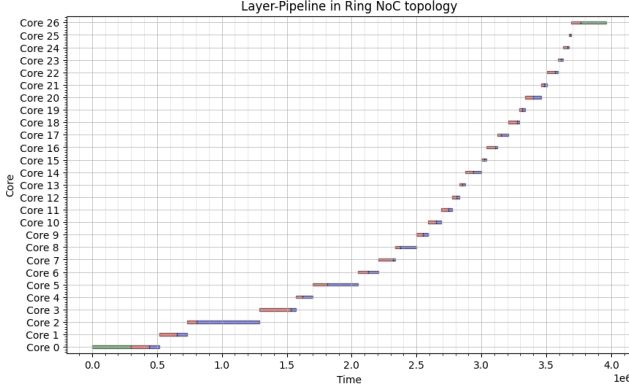


Figure 10. MobileNet on 27 cores with LP and 'os' dataflow. Compute(Red), NoC(Blue), DRAM(Green)

Now, shifting our focus to the examination of various NoC topologies in the context of MobileNet on 27 cores with LP and output-stationary dataflow, we explore different configurations such as ring, mesh 3x9, mesh 9x3, hypercube, and switch. For instance, in our setup, core<n> sends data to core<n+1>. In the ring topology, each transfer occurs on exactly one link. In contrast, in a 3x9 mesh topology, most transfers utilize one link, but a few transfers on the last column require nine links to reach the core on the opposite side. This demonstrates that the mesh 3x9 topology performs worse than the ring topology. Additionally, proper scheduling can optimize link utilization even in mesh topologies, emphasizing the interconnect of scheduling and NoC topology.

The analytical model graphs and table 1 also shed light on another optimization opportunity. We observe certain cores performing significantly more work compared to others, as illustrated in figure 10. To address this imbalance, we can implement intra-layer scheduling, grouping a few cores to handle specific heavy layers. This approach results in a combination of intra- and inter-layer scheduling, further enhancing performance and average mapping efficiency. Our integration of ANoC in Kritika enables such optimizations in Kritika, architects gain the flexibility to experiment with different scheduling strategies. This example underscores the potential of NoC support in Kritika for optimizing DNN simulations.

4.1.3 ANoC Integration with Kritika. Navigating the integration between Anoc and Kritika posed an initial challenge owing to their differing coding languages, with Anoc constructed in C++ while Kritika was developed in Python.

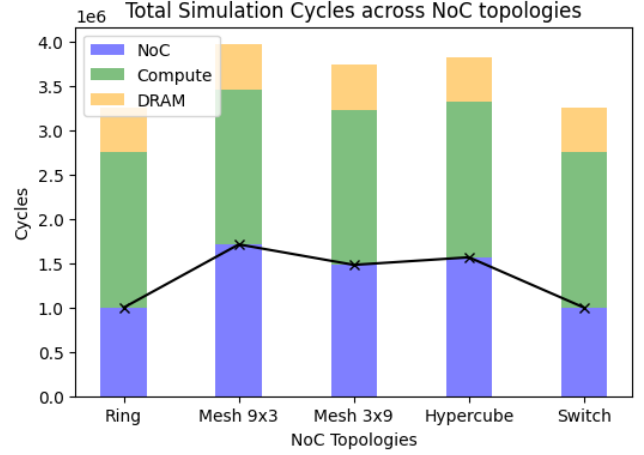


Figure 11. MobileNet on 27 cores with LP and 'os' dataflow simulated on various NoC topologies

MobileNet on 27 cores				
Scheduling scheme	Total Cycles	NoC	DRAM	Compute
LS (multi-port)	1,822,126	0	88.89%	11.11%
LS (single-port)	43,935,414	0	99.51%	0.49%
LP	3,963,672	43.17%	12.66%	44.17%

Table 1. NoC, DRAM, Compute contributions in LS and LP

To Overcome this disparity, we employed the Cythonize module, which helped in building a cohesive binary that amalgamated both the platforms. This fusion allowed Kritika to access all the functionalities native to Anoc. Anoc uses a multifaceted approach for topology management, it breaks down the topology into distinct devices and their connections as links. In modeling the network topology, Anoc implements several key classes tailored to efficiently represent the intricate architecture:

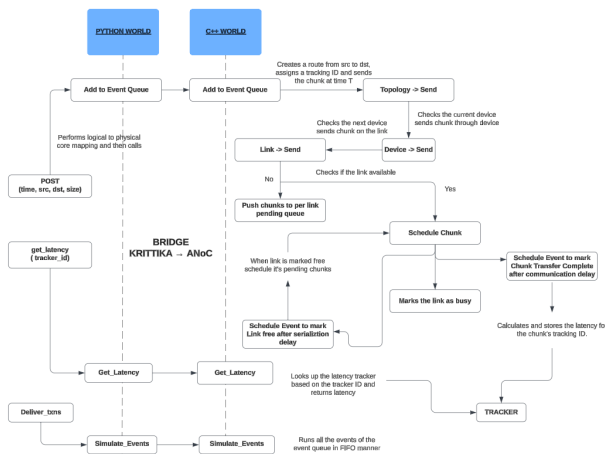
- **Devices:** In our context, these correspond to cores, serving as fundamental units within the network topology.
- **Links:** This class specifies the connections between devices, crucial for delineating the topology structure and facilitating data transfer.
- **Chunk:** An essential component for modeling data transmission across devices, providing a framework for understanding the flow of information within the network.

Anoc further distinguishes between two types of networks: congestion unaware and congestion aware. For congestion unaware simulations, the system simplifies calculations by determining the hops required for chunk traversal and subsequently computes the final latency based on the latency associated with each link. In contrast, for congestion-aware

scenarios, Anoc adopts a more sophisticated approach. It relies on an event queue mechanism, where every chunk transfer initiates a series of events, such as reserving a link, marking a link as free, and signaling the completion of the transfer. During simulation, when a transfer is scheduled to occur, Anoc first checks if the required link is available. If it is, the system proceeds by marking the link as busy and scheduling events to mark the link as free after serialization delay, as well as to mark the chunk transfer complete after the communication delay.

$$\text{Serialization delay} = \frac{\text{chunk_size}}{\text{bandwidth_Bps}}$$

However, if the link is already in use, the event is added to a pending queue. These pending events are subsequently re-attempted for scheduling once the link becomes available, triggered by the completion of other events within the system. This intricate event-driven approach enables Anoc to effectively model congestion and the bottlenecks that arise when different devices try to compete for the same set of resources.



An essential aspect to highlight is that congestion modeling within Anoc operates on the principle of scheduling all data transfers beforehand, followed by simulating the entire event queue in a first-in-first-out (FIFO) manner. This methodology stems from the intrinsic nature of time progression within the system: time doesn't advance continuously but rather is event-driven. In this framework, each event in the queue triggers the execution of a corresponding callback function. These functions are pivotal as they not only schedule additional events but also execute various operations or call other functions within the system. Crucially, upon processing an event, the system updates the current time to reflect the completion time of that event. This approach

The disparity in temporal synchronization between Krittika and Analytical Noc posed a significant challenge in establishing a cohesive notion of time across the systems. Addressing this challenge was necessary because otherwise we would not be modelling congestion that occurs during the communication effectively. So to address this challenge, we opted to rely on event-driven system which already exists in ANoC. Rather than simply appending related events to the queue during instances of communication by Krittika, we implemented a nuanced approach. Specifically, we introduced an additional layer of scheduling wherein an auxiliary event was programmed to occur at time T which is the time at which krittika is trying to communicate and completion of this event leads to original events related to communication to be added to the event-queue. This methodology helped us to harmonize the temporal progression across both Krittika and Analytical Noc systems, thereby mitigating temporal disparities and enabling us to model congestion accurately.

Since we plan to enable the performance analysis of a larger variety of multi-core accelerators, which in turn necessitates support for a broader network topology, we further extended ANoC support by adding three new topologies - namely Bus, Mesh and Hyper Cube. AstraSim supports new topologies through the easily extendable BasicTopology class. Any new topology needs to initialize the links between each device according to the topology, and setup this device-to-link map in each device as well. For instance, like most other topologies, Mesh and Hyper Cube required setting up new links for each connection between a pair of devices. However, Bus just required creating and using a ptr to the same link for due to the structure of the bus topology. Fig. 13 illustrates these topologies using a sample number of cores.

Finally we add support for configuring the logical to physical mappings of cores on the network on chip. Firstly, it provides flexibility to adapt to diverse communication patterns and data flow requirements across different applications. Additionally, the ability to switch between mappings enhances fault tolerance and reliability, ensuring system functionality in the presence of faults. Overall, this capability enhances the versatility, performance, fault tolerance, scalability, and adaptability of the NoC model, making it a valuable tool for designing complex communication architectures in modern computing systems.

4.1.4 Modeling SIMD Operations. A detailed review of the existing SIMD implementations in *Krittika* was done to get an understanding of where new SIMD operations can be integrated. This review was a great insight into the capabilities and limitations for forming the basis for the subsequent enhancements. We used multiple non-linear functions to

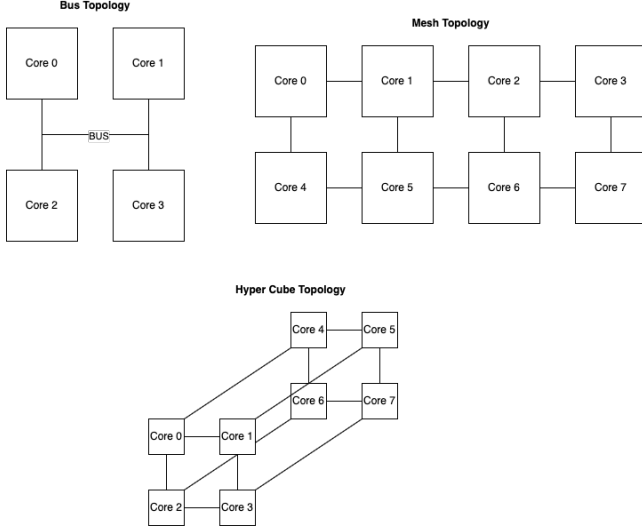


Figure 13. Sample illustration of newly added topologies

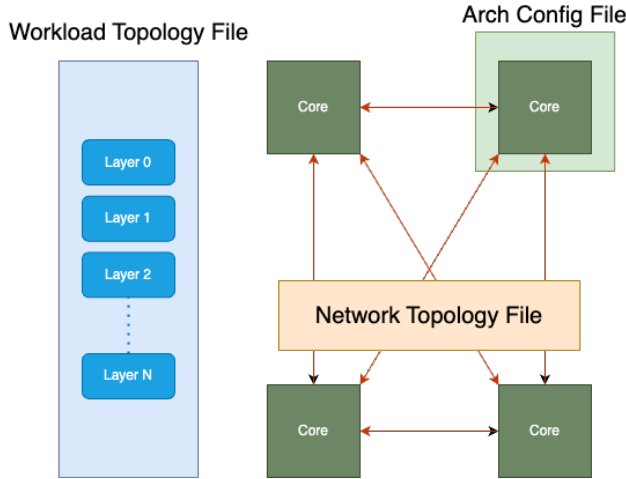


Figure 14. Summary of user inputs to modified Krittika

profile SIMD, that are compatible with AlexNet and BERT like $\tanh$, ReLU, GeLU, SeLU, Sigmoid, Softmax, Swish, Mish, Softplus, softsign, and Sinusoid.

A robust RISC-V development environment was set up using Rocket Core [4], incorporating necessary tools and libraries for RISC-V development and simulation. Initially, a RISC-V GNU Toolchain was installed and this was the basis for installing all other RISC-V simulator necessities. This is essential for properly testing and profiling new SIMD operations to ensure that improvements are properly validated in a controlled simulation environment.

- Based on the RISC-V GNU Toolchain [9], other RISC-V simulator programs such as the Spike RISC-V Simulator [6], RISC-V Proxy Kernel [5] which is used to run test files on the Spike simulator were installed.

- Each of these additional programs are independent of the Toolchain, i.e., the working of these are not dependent on that of the Toolchain as long as the Toolchain is installed properly.

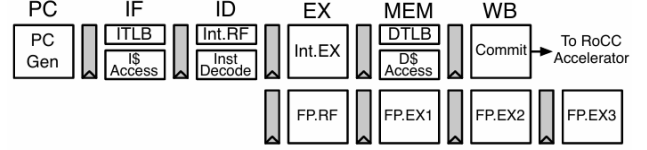


Figure 15. The Rocket Core Pipeline

A critical component of modeling and profiling SIMD operations was writing benchmark test files, targeted for each operation. The benchmark files were run on the Spike RISC-V simulator.

- The benchmark was implemented in C and compiled using the RISC-V GCC compiler, optimized for performance with appropriate architecture flags.
- Using the Spike RISC-V ISA simulator, the benchmark files of all the non-linear functions were executed, and detailed profiling was conducted.
- Each of the functions was tested on a large set of randomly generated double-precision floating-point values to mimic a realistic workload.

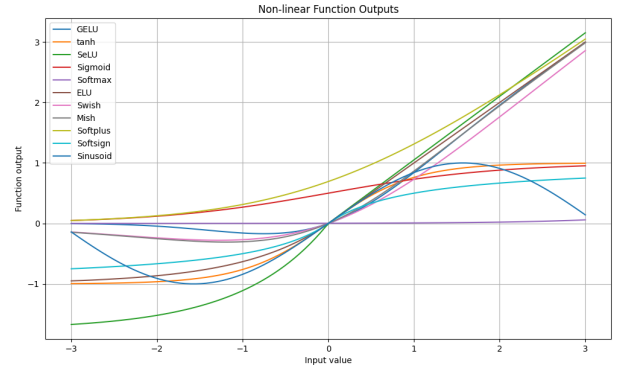


Figure 16. Variation of Non-Linear Functions for sample input

4.2 Evaluation

4.2.1 Planned Analyses. Our proposed extensions to *Krittika* are general, so any combination of the implemented SIMD operators, network topologies, and partitioning scheme will work and give cycle-accurate performance metrics. However, for the sake of our analysis, we intend to evaluate AlexNet and BERT. Using these models, we plan to present an analysis of the impact of different network topologies (such as ring, fully connected, bus, etc.), the effect of different inter-core network parameters (such as bandwidth

and latency), the impact of our chosen partitioning scheme, and the effect of using the NoC for inter-SRAM and SRAM-DRAM communication.

4.2.2 Validation. Since we are extending *Krittika*, we also need to ensure that we do not regress the functional correctness and performance accuracy that exists in the simulator already. To verify the functional correctness of the partitioning and final output distributed across multiple cores, we plan to use the output trace .csv file dumped and compare this against a Python computation of the inputs. We will also compare the performance we get from the simulator with our analysis. Finally, we will also write multiple unit tests across the codebase to verify our functionality at a smaller scale.

5 Results

We enhanced *Krittika* by integrating ANoC and adding more SIMD units. This upgrade allowed users to efficiently run large-scale models like AlexNet and BERT. Utilizing our available computing resources, we successfully simulated GEMM layers across 4 to 10 cores. The configuration used in our experiments, including SCALE-SIM and ANoC settings, is detailed in Table 2. This comprehensive setup enabled us to conduct rigorous evaluations and analyze the performance of *Krittika* with ANoC integration. As discussed before we present all the analysis on Output Stationary Dataflow. Note a very high active buf fraction has been used in IFMAP and OFMAP to reduce the effect of prefetching, as NoC sends the data over, prefetching has to be effectively disabled. However, for Filter maps it's not required so a default value of 0.5 is being used. We limit our analysis to GEMM operations of having 100x100x100 due to the limitations of the compute resources. We use 8 cores/ 8 layers to simulate results (Unless mentioned otherwise)

Configuration	
SRAM Sizes (KB)	5
DRAM Read latency (Cycles)	100
DRAM Write latency (Cycles)	200
NoC Hop latency (Cycles)	5
NoC BW (GBPS)	50
PE array	4x4
Per core user buf interface BW (words/cycle)	5
DataFlow)	OS
IFMAP and OFMAP Active buf frac)	0.999
Filter Map buf frac)	0.5

Table 2. Parameters used in SCALE-SIM and Analytical NoC

Layer-Pipeline versus Layer-Sequential With NoC support in *Krittika*, we can implement the layer pipeline

scheduling scheme. This means that the output maps are transferred over the NoC to the next cores, which then act as the input maps, thereby reducing the need for DRAM write and read operations. Figure 17 illustrates how layer-pipeline scheduling outperforms layer-sequential scheduling. To highlight this difference, we adjust the DRAM latency cycles and NoC bandwidth accordingly in the configuration. The flexibility of the simulator's configuration file approach allows us to experiment with different parameters, catering to the Design Space Exploration (DSE) on NoC and DRAM bandwidth.

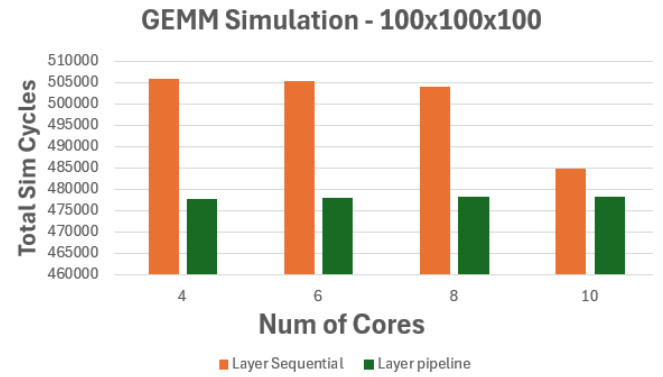


Figure 17. LS v/s LP across 4 to 10 cores

Layer-Sequential with a Single DRAM Port In multi-core systems, having a dedicated DRAM port for each core is impractical. We discussed earlier how NoC becomes necessary to facilitate DRAM interactions. Figure 18 explores the impact of different NoC topologies on layer-sequential scheduling. For example, in a ring topology where core0 is connected to DRAM, each core2 needs to traverse additional links to access DRAM(Depending on the topology). The computational workload remains constant across configurations, as does the DRAM cycle count since there's no data reuse. However, the total cycle count increases due to the overhead introduced by NoC.

Congestion-Aware NoC The Analytical NoC model offers two simulation scenarios: congestion-unaware and congestion-aware. We examine layer-pipeline scheduling across different NoC topologies under both modes. Figure 19 illustrates how congestion-aware simulations result in longer cycles compared to congestion-unaware ones. This increase in cycle count is due to the additional time taken to manage and mitigate network congestion, which leads to delays in data transmission and processing.

NoC Bandwidth The flexibility of the configuration file allows users to generate different settings for experimentation. In figure 20, we observe how variations in NoC bandwidth affect total simulation cycles and the contribution of

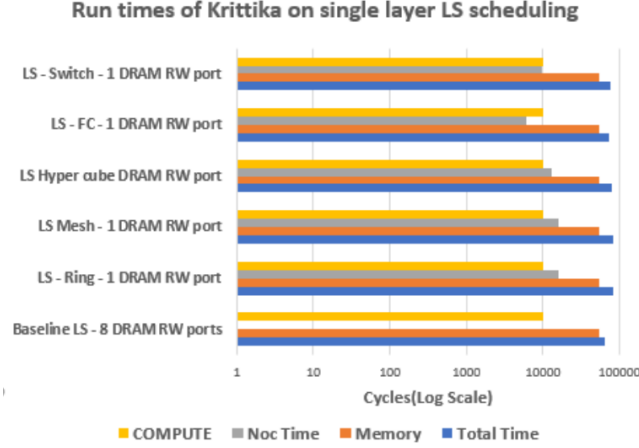


Figure 18. LS in single port DRAM setting

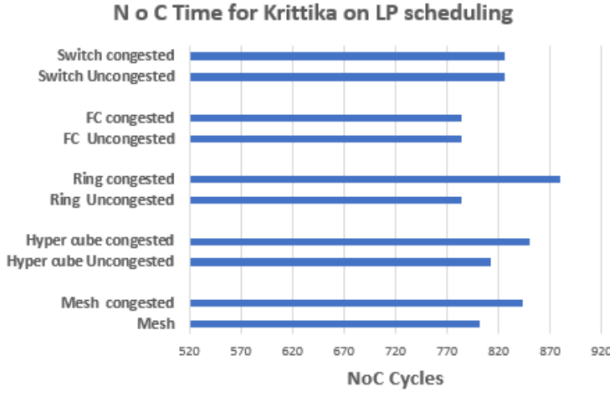


Figure 19. NoC congestion aware and unaware with LP

NoC to these cycles. As anticipated, increasing NoC bandwidth reduces total simulation cycles. However, this reduction reaches a saturation point when compute and/or DRAM operations become dominant factors. Such insights can aid architects in determining optimal NoC bandwidth without necessitating additional resources.

SIMD Profiling The latency for each activation function are presented in table 3 based on the results of the compilation.

6 Conclusion

In conclusion, through this work, we have further enabled the exploration of large-scale DNN architectures on multi-core systolic array based accelerators using Krittika with the following key contributions and provide the repo here [11]:

Analytical NoC model integration: Since almost all multi-core systolic array based accelerators utilize a NoC for inter-core communication, by successfully integrating a NoC model into Krittika and consequently factoring for this communication and network delay in the performance

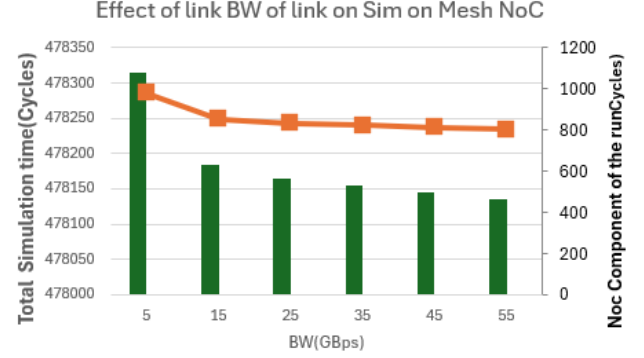


Figure 20. 8 core / 8 layer run on LP scheme showing the effect of link BW on total cycles

Operation	Latency(s)
<i>tanh</i>	0.000094
ReLU	0.000008
GeLU	0.000025
SeLU	0.000003
Sigmoid	0.000045
Softmax	0.000001
Swish	0.000006
Mish	0.000025
Softplus	0.000647
Softsign	0.0000096
Sinusoid	0.000011

Table 3. Latencies of SIMD Operators

reports generated by the simulator, we further the usefulness of Krittika for early accelerator exploration and evaluation.

New topologies in AstraSim’s ANoC model: We added support for three new topologies (namely Bus, Mesh and Hyper Cube) in the ANoC model developed as part of AstraSim. As can be seen from my results section above, by using these topologies, accelerators with different network topologies can be evaluated using Krittika.

Incorporating LS and LP scheduling: We modified Krittika to add support two partitioning schemes namely Layer Sequential and Layer Pipelined. Besides using these schemes for performance studies, these changes also pave the way for adding support for any partitioning scheme in the future.

Profiling SIMD units: Our changes add support in Krittika for a diverse set of SIMD activation functions while still being performance-accurate through the profiling results. Thus, Krittika can be used for a greater variety of DNN architectures that was previously not possible due to lack of activation function support.

7 Collaboration Plan

7.1 Implementing Partitioning Schemes

Tasks:

- Implement LP scheduling with tile interleaving approach to extract maximum pipelining
- Add support for LS scheduling (single layer) to send traffic to single node to simulate congestion.
- Adding support to generate cycle-accurate traces for both modified LS and LP scheduling
- Implementing a traceless analytical model for light-weight exploration and reference in the *Krittika* code-base, depicting expected trends with Analytical NoC integration when simulating over 10 cores

Team members involved

1. Manish Manchali
2. Lohith Artham

7.2 Extending SIMD support in Krittika

Tasks:

- Profile and get latency estimates of SIMD units using the RISC-V simulator
- Integrate support for these SIMD operators in *Krittika*.

Team members involved

1. Suryaa Senthilkumar Shanthi

7.3 Adding NoC model support to Krittika

Tasks:

- Setup python to C++ bindings using Cython between *Krittika* and ANoC
- Implement required functionality and APIs in ANoC
- Setup Python APIs in *Krittika* corresponding to NoC transactions
- Implement new topologies such as Mesh, Bus and Hyper Cube.

Team members involved

1. Somesh Nawandar
2. Sreemanth Prathipati

References

- [1] Niket Agarwal, Tushar Krishna, Li-Shiuan Peh, and Niraj K Jha. Garnet: A detailed on-chip network model inside a full-system simulator. In *2009 IEEE international symposium on performance analysis of systems and software*, pages 33–42. IEEE, 2009.
- [2] Manoj Alwani, Han Chen, Michael Ferdman, and Peter Milder. Fused-layer cnn accelerators. In *2016 49th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, pages 1–12. IEEE, 2016.
- [3] Yuhao Zhu Paul Whatmough Matthew Mattina Tushar Krishna Ananda Samajdar, Jan Moritz Joseph. A systematic methodology for characterizing scalability of dnn accelerators using scale-sim. *ISPASS: International Symposium on Performance Analysis of Systems and Software*, 2020.
- [4] Krste Asanović, Rimas Avižienis, Jonathan Bachrach, Scott Beamer, David Biancolin, Christopher Celio, Henry Cook, Palmer Dabbelt, John Hauser, Adam Izraelevitz, Sagar Karandikar, Benjamin Keller, Donggyu Kim, John Koenig, Yunsup Lee, Eric Love, Martin Maas, Albert Magyar, Howard Mao, Miquel Moreto, Albert Ou, David Patterson, Brian Richards, Colin Schmidt, Stephen Twigg, Huy Vo, and Andrew Waterman. The rocket chip generator. Technical Report UCB/EECS-2016-17, EECS Department, University of California, Berkeley, April 2016.
- [5] aswaterman. Riscv-pk. <https://github.com/riscv-software-src/riscv-pk/tree/master>, 2013.
- [6] aswaterman. Riscv-isa-sim. <https://github.com/riscv-software-src/riscv-isa-sim>, 2017.
- [7] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. *arXiv preprint arXiv:1810.04805*, 2018.
- [8] Mingyu Gao, Xuan Yang, Jing Pu, Mark Horowitz, and Christos Kozyrakis. Tangram: Optimized coarse-grained dataflow for scalable nn accelerators. In *Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems*, pages 807–820, 2019.
- [9] Kito-Cheng. Riscv-gnu-toolchain. <https://github.com/riscv-collab/riscv-gnu-toolchain>, 2016.
- [10] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E Hinton. Imagenet classification with deep convolutional neural networks. *Advances in neural information processing systems*, 25, 2012.
- [11] sree. Krittika- updated noc and simd. https://github.com/5ree/krittika_hml_proj, 2024.
- [12] William Won, Taekyung Heo, Saeed Rashidi, Srinivas Sridharan, Sudarshan Srinivasan, and Tushar Krishna. Astra-sim2.0: Modeling hierarchical networks and disaggregated systems for large-model training at scale, 2023.
- [13] Hsin-I Wu, Da-Yi Guo, Hsu-Hsun Chin, and Ren-Song Tsay. A pipeline-based scheduler for optimizing latency of convolution neural network inference over heterogeneous multicore systems. In *2020 2nd IEEE International Conference on Artificial Intelligence Circuits and Systems (AICAS)*, pages 46–49. IEEE, 2020.